



COM3523 Software Reengineering

Mini Report

UG19

2nd May 2025

NAME: Randi Chandrasena

- Created the and wrote the mini report, pushed it to the repository

NAME: Manya Mittal

- Did the refactoring changes

NAME: Jay Harrison

- Did the refactoring changes

QUESTIONS TO CONSIDER:

- What refactoring techniques did you apply?
- What was the reason for applying these techniques?
- How do you know that observable behaviour has not changed after refactoring?

Regression testing – we ran the tests and they all passed successfully.

- What are the (potential) benefits and drawbacks of applying these techniques?
- Overall, how do you feel about the refactoring process? Worth it? Why (or why not)?

Changes We Made:

1. Inline Variable instead of heavy_item in calculate_shipping_fee.

43	43	-	def calculate_shipping_fee(self, purchases):
44	-	-	heavy_item = False
45	44	-	for purchase in purchases:
46	45	-	if purchase.get('weight', 0) > 20:
47	-	-	heavy_item = True
48	-	-	break
46	+	+	return 50
47	+	+	return 20

47	-	-	heavy_item = True
48	-	-	break
46	+	+	return 50
47	+	+	return 20

49	48	-	
50	-	-	if heavy_item:
51	-	-	return 50
52	-	-	else:
53	-	-	return 20
54	-	-	
55	-	-	def calculate_shipping_fee_for_heavy_items(purchases):
56	-	-	for purchase in purchases:
57	-	-	if purchase.get('weight', 0) > 20:
58	-	-	return 50
59	-	-	return 20

49	+	+	# def calculate_shipping_fee_for_heavy_items(purchases):
50	+	+	# for purchase in purchases:
51	+	+	# if purchase.get('weight', 0) > 20:
52	+	+	# return 50
53	+	+	# return 20

2. Move function add_purchases to add_purchase.

14	14		<code>def add_purchase(self, name, purchase):</code>
15	15		<code>self.add_customer(name, [purchase])</code>
16	16		
17	-		<code>def add_purchases(self, name, purchases):</code>
18	-		<code>self.add_customer(name, purchases)</code>
17	+	#	<code>def add_purchases(self, name, purchases):</code>
18	+	#	<code>self.add_customer(name, purchases)</code>

3. Inline variable for fragile_item in calculate_shipping_fee_for_fragile_items

55	55		<code>def calculate_shipping_fee_for_fragile_items(purchases):</code>
56	-		<code>fragile_item = False</code>
57	56		<code>for purchase in purchases:</code>
58	57		<code>if purchase.get('fragile', False):</code>
59	-		<code>fragile_item = True</code>
60	-		<code>break</code>
61	-		<code>if fragile_item:</code>
62	-		<code>return 60</code>
63	-		<code>else:</code>
64	-		<code>return 25</code>
58	+		<code>return 60</code>
59	+		<code>return 25</code>

4. Removed redundant variable flat_tax

56	56	for purchase in purchases:
57	57	if purchase.get('fragile', False):
58	58	return 60
59	-	return 25
60	-	
61	-	flat_tax = 0.2
	⊖	
59	+	return 25

5. Renamed 'a' in generate_report to total_price

20	20	def generate_report(self):
21	21	for y, x in self.customers.items():
22	-	a = 0
22	+	total_price = 0
23	23	for z in x:
24	24	if z['price'] > self.tax_threshold:
25	25	taxed_price = z['price'] * (1 + self.tax_rate)
26	-	a += taxed_price
26	+	total_price += taxed_price
27	27	else:
28	-	a += z['price']
28	+	total_price += z['price']
29	29	print(y)
30	-	if a > self.discount_threshold:
30	+	if total_price > self.discount_threshold:
31	31	print("Eligible for discount")
32	32	else:
33	-	if a > 300:
33	+	if total_price > 300:
34	34	print("Potential future discount customer")
35	35	else:
36	36	print("No discount")
37	-	if a > 1000:
37	+	if total_price > 1000:
38	38	print("VIP Customer!")
39	39	else:
40	-	if a > 800:
40	+	if total_price > 800:
41	41	print("Priority Customer")